Project Number 1 AI Harness In C Standard library.

Core State Machine.

Key requirements:
This is going to be a command line.
You Can only use C Standard libraries I.e. <stdio.h> and <string.h>
This should run using an infinite while loop. Only exit when prompted.
When you type “exit” The loop should end and it should go into the exit state where it clears any memory and says goodbye. Then program ends.
Assume 5 turns means 10 different chats. Meaning 1 chat from human then 1 chat from AI is equal to 1 turn.

State Running
- Prompt user (“>”)
- We only want to use the C Standard Library. Use fgets [fgets(buffer,256,stdin)]
- We always want to strip the newline character.
- If we want to exit the state machine we look for the keyword “exit”
    - Use Strcmp(buffer, “exit”) == 0; We will go to STATE: Exit;
    - Pass buffer to Storage_Context();
    - Store buffer into Storage[Storage_Agent_Idx][254]
- We will pass the buffer to the function Model().
    - Store user message into Storage
    - Print the model function output and stay in the running state.
    - Store model response in Storage[Storage_agent_idx][254]

State Exit:
- Free all allocated memory to prevent memory leaks.
- Change context management state to END State.
- Print “Goodbye.”
- Exit(0)

Context Management.
- To keep memory free we must limit ourselves to the last 5 turns.
- There should be 3 states in this state machine.
- STATE less than 5 turns
    - Loads buffer passes buffer register to the 2D array. Storage[9][255];
    - There should be a variable Storage_Idx that increments upward towards 10.
    - Storage_Agent_Idx = Storage_Idx
    - If storage_Idx = 10 then flip States to CONTEXT_Window.
- State CONTEXT_WINDOW
    - Clear Stroage[Storage_Idx = 0][254].
    - Clear the string with null characters.

        - Shift all of the strings in Storage up 1 so that context Can be loaded at Storage_Idx = 9 and off loaded or freed at Storage_Idx = 0. This follows the FIFO (First In First Out principle.
- State END
    - For every Storage_Idx Clear the string to start fresh. So clear with null characters.
    - Set storage_Idx = 0;
    -

State Running

- Prompt user (“>”)
- We only want to use the C Standard Library. Use fgets [fgets(buffer,256,stdin)]
- We always want to strip the newline character.
- We check for the keyword “exit” FIRST, before we store anything.
    - Use strcmp(buffer, “exit”) == 0; We will go to STATE: Exit;
    - Do NOT pass this buffer to Storage_Context(). We don’t want “exit” sitting in our history.
- If it’s not “exit”:
    - Pass buffer to Storage_Context(buffer, USER);
    - Store user message into Storage[Storage_Idx][256]
    - We will pass the buffer to the function Model().
    - Store model response into Storage via Storage_Context(response, MODEL);
    - Print the model function output and stay in the running state.

State Exit:

- Our Storage array is static/global, not malloc’d — so we don’t actually free() anything here, we just clear it out.
- Change context management state to END State.
- Print “Goodbye.”
- Exit(0)

Context Management.

- To keep memory free we must limit ourselves to the last 5 turns.
- Storage[10][256]; — 10 slots since 5 turns = 10 messages (1 user + 1 model per turn).
- Storage_Idx starts at 0 and increments upward toward 10 on every single write.
- Storage_Agent_Idx is no longer the same thing as Storage_Idx — it tags WHO wrote the message (USER or MODEL) so we know whose turn it was.

Storage_Context(buffer, Storage_Agent_Idx):

- If Storage_Idx = 10, we run the CONTEXT_WINDOW shift BEFORE we write anything new.
- Write buffer into Storage[Storage_Idx].
- Tag it with Storage_Agent_Idx so we remember who said it.
- If Storage_Idx < 10, Storage_Idx++.

State CONTEXT_WINDOW:

- Only triggers when Storage_Idx = 10, right before the new write.
- Shift all of the strings in Storage up 1 so that context can be loaded at Storage_Idx = 9 and off loaded at Storage_Idx = 0. This follows the FIFO (First In First Out) principle.
- Clear Storage[9] with null characters before the new message lands there.
- Storage_Idx just stays at 10 from here on — we shift instead of incrementing.

State END

- For every Storage_Idx clear the string to start fresh. So clear with null characters.
- Set Storage_Idx = 0;

Tool and Parsing behaviour.

Calculator tool.

We will do simple calculations.

Addition, Subtraction, Multiplication, and division.

We will look for synonyms for Addition, Subtraction, Multiplication, Division, and Calculate. Assume an input response will be “Calculate 2 + 2” it will need to be able to assign “+,-,*,/, to their specified operators. The output from the tool should be the calculated value. Keep it as simple as possible. This tool should pass its response to the AI and back to the user. The response from the tool should be added to storage.

Tool Integration: Calculator

State Tool_Calculator

- We will support simple calculations: Addition, Subtraction, Multiplication, and Division.
- To keep parsing simple, we require a strict input format: "calculate NUM OP NUM" (e.g. "calculate 2 + 2"). We are NOT supporting multi-step expressions like "2 + 2 * 3" — only two numbers and one operator.
- We look for a trigger keyword like "calculate" (and synonyms) inside the user's buffer to decide if the tool should run instead of the normal Model() echo/hello logic. This check

must happen BEFORE the "hello" check in Model(), otherwise a calculation request could get swallowed by the echo/hello logic.

- We also look for synonyms of each operation so the user doesn't have to type exact symbols:
    - Addition → "add", "plus", "sum" → maps to '+'
    - Subtraction → "subtract", "minus", "less" → maps to '-'
    - Multiplication → "multiply", "times", "product" → maps to '*'
    - Division → "divide", "divided by", "over" → maps to '/'
- Parse the buffer:
    - Use strtok or sscanf to split the buffer into tokens (numbers and operator words/symbols), since we're enforcing the strict format above.
    - Convert number tokens using sscanf directly (e.g. sscanf(token, "%d", &num)) — NOT atoi/atof, since those require <stdlib.h>.
    - Match operator tokens against our synonym list to figure out which operator to actually perform.
- Perform the calculation based on the matched operator.
    - Divide by zero check: if the operator is '/' and the second number is 0, do NOT perform the division. Return an error string like "Error: divide by zero" instead.
- Store the calculated value (or the divide-by-zero error string) as a string (snprintf into a buffer) so it can go through the same Storage/Model pipeline as everything else.
- Function signature: int Tool_Calculator(char *input, char *output); — takes the input buffer, writes the result into the output buffer, matching the same "pass buffer in, write buffer out" pattern as Model().

Flow (Tool → Model → User → Storage):

1. Tool_Calculator(buffer, output) runs, writes result string into output (this includes the divide-by-zero error case).
2. Output string gets passed into Model() so it can be framed as part of the AI's response (e.g., "The answer is 4").
3. Model's output (containing the tool result) prints to the user like normal.
4. Model's output is stored into Storage via Storage_Context(response, MODEL) — same as any other model response.